

Trabajo Práctico — Algoritmos

Sistema de Bicicletas Públicas

Descripción general

El sistema debe gestionar una red de estaciones de bicicletas públicas de la ciudad de Buenos Aires: las estaciones, las bicicletas, los viajes que hacen los usuarios y la redistribución de bicicletas entre estaciones. Se implementa en C++ con orientación a objetos, usando memoria dinámica y las estructuras de datos vistas en la cursada.

Las estructuras que pide cada sección (tabla de hash, cola, heap, árbol, grafo) se implementan a mano: no se pueden usar `vector`, `map`, `unordered_map`, `queue`, `priority_queue` ni `set` de la STL. Sí se permite usar `std::string` y la STL para leer archivos y para tareas auxiliares.

Archivos de entrada

Cada archivo tiene un registro por línea, con los campos separados por espacios. Se puede asumir que los archivos están bien formados.

estaciones.txt

```
codigo nombre barrio capacidad_anclajes bicis_disponibles
```

Ejemplo:

```
PLZ PlazaDeMayo Monserrat 30 12
PAL ParqueTresDeFebrero Palermo 40 35
CAB PrimeraJunta Caballito 25 4
```

bicicletas.txt

```
bici_id codigo_estacion tipo estado kilometros
tipo es Mecanica o Electrica. estado es OK o Averiadada.
```

Ejemplo:

```
501 PLZ Mecanica OK 1240
502 PAL Electrica Averiadada 3810
```

viajes.txt

```
viaje_id bici_id dni_usuario estacion_origen estacion_destino fecha duracion_min
Las fechas tienen formato AAAAMMDD.
```

Ejemplo:

```
9001 501 30123456 PLZ PAL 20260310 25
9002 502 27654321 PAL CAB 20260311 40
```

ciclovias.txt

Indica qué estaciones están unidas directamente por una ciclovía. Las conexiones son bidireccionales.

estacion_a estacion_b

Ejemplo:

```
PLZ CAB
CAB PAL
```

A. Gestión de estaciones

- Mostrar la información de una estación dado su código.
- Agregar una estación nueva al sistema.
- Eliminar una estación existente. No hace falta reubicar sus bicicletas.
- Listar todas las estaciones ordenadas por capacidad de anclajes o por bicicletas disponibles, a elección del usuario. El algoritmo de ordenamiento se implementa a mano: burbujeo, selección o inserción.
- Listar las estaciones con menos del 20% de ocupación (bicis disponibles sobre capacidad).
- Dadas dos estaciones, calcular el recorrido por ciclovías con **menor cantidad de tramos** usando búsqueda en anchura (BFS). Si no existe recorrido, informarlo.

Requisito estructural: indexar las estaciones por código en una tabla de hash con resolución de colisiones. Usar esta función hash que crea conveniente.

Indicar en el README cuántas colisiones hubo con los datos de prueba y cuál es el factor de carga final.

B. Viajes y colas de espera

1. Calcular la cantidad de viajes que salieron de una estación dentro de un rango de fechas.
2. Buscar todos los viajes de un usuario dado su DNI y mostrar el total de minutos pedaleados.
3. Listar las bicicletas con más de X kilómetros, donde X lo ingresa el usuario, para enviarlas a revisión.
4. Implementar una **cola** de espera por estación: si un usuario quiere retirar una bici y no hay disponibles, se encola. Cuando se devuelve una bici en esa estación, se le asigna al primero de la cola.
5. Implementar una **cola de prioridad (min-heap)** para el taller de reparaciones. Cada bicicleta averiada entra con una prioridad del 1 (más urgente) al 3; las eléctricas entran siempre con prioridad 1. Solo se piden dos operaciones: insertar una bici y extraer la más urgente.

Consigna de análisis: para cada punto, indicar la complejidad temporal en notación Big O y explicar en dos o tres oraciones por qué se eligió esa estructura de datos.

C. Registro de bicicletas con árbol binario de búsqueda

Todas las bicicletas de la red se guardan en un árbol binario de búsqueda (ABB) ordenado por `bici_id`.

- Insertar una bicicleta en el árbol.
- Buscar una bicicleta por ID y mostrar sus datos.

- Listar todas las bicicletas ordenadas por ID (recorrido inorder).
- Contar cuántas bicicletas averiadas hay, recorriendo el árbol.
- Calcular la altura del árbol.
- Liberar toda la memoria del árbol al finalizar el programa (recorrido postorder).

Pregunta de análisis: ¿qué altura tendría el árbol si las bicicletas se cargaran ordenadas por ID? ¿Cómo afecta eso al costo de la búsqueda? Responder con un ejemplo de 5 bicicletas.

Opcional (suma puntos): eliminar una bicicleta del árbol, contemplando los tres casos (nodo hoja, con un hijo, con dos hijos).

D. Redistribución con backtracking

Un camión recorre la ciudad para llevar bicicletas a las estaciones vacías. Tiene lugar para exactamente C bicicletas y debe descargarlas completas. Hay que decidir a qué estaciones ir.

- **Entrada:** la capacidad C del camión y una lista de estaciones con la cantidad de bicicletas que le faltan a cada una.
- **Salida:** todos los subconjuntos de estaciones cuyos faltantes suman exactamente C. Si no hay ninguno, informarlo.
- Implementar la solución con backtracking.
- Indicar la complejidad temporal en el peor caso.

Ejemplo: con $C = 10$ y faltantes PLZ 3, CAB 7, RET 5, CON 2, BOE 5, las soluciones son {PLZ, CAB}, {RET, BOE}, {PLZ, RET, CON} y {PLZ, CON, BOE}.

Condiciones de entrega

- Exposición de avances cada dos semanas, con el código funcionando.
- Entrega final con defensa oral del proyecto.
- Diagrama de clases UML con las relaciones entre clases (asociación y composición).
- Menú por consola que permita probar cada funcionalidad.
- Variables y métodos en camelCase; clases en PascalCase.
- Las clases deben ser originales y propias. La detección de copias resultará en la pérdida de la regularidad de todos los involucrados.
- Archivo README.md con instrucciones de compilación, descripción de cada módulo y las respuestas a las consignas de análisis.
- El programa debe compilar sin warnings con `g++ -std=c++17 -Wall -Wextra` y no debe tener pérdidas de memoria (se recomienda verificar con Valgrind).